

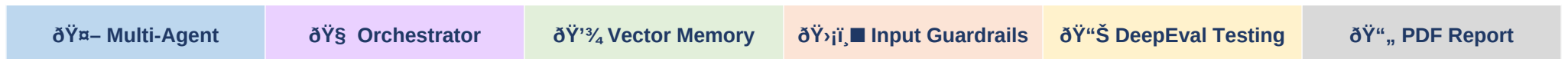
Multi-Agent Trip Planner

Built with LangGraph - Memory - Orchestrator - PDF Output

Author: Kavin K - Course: AFDE Prodapt Chennai

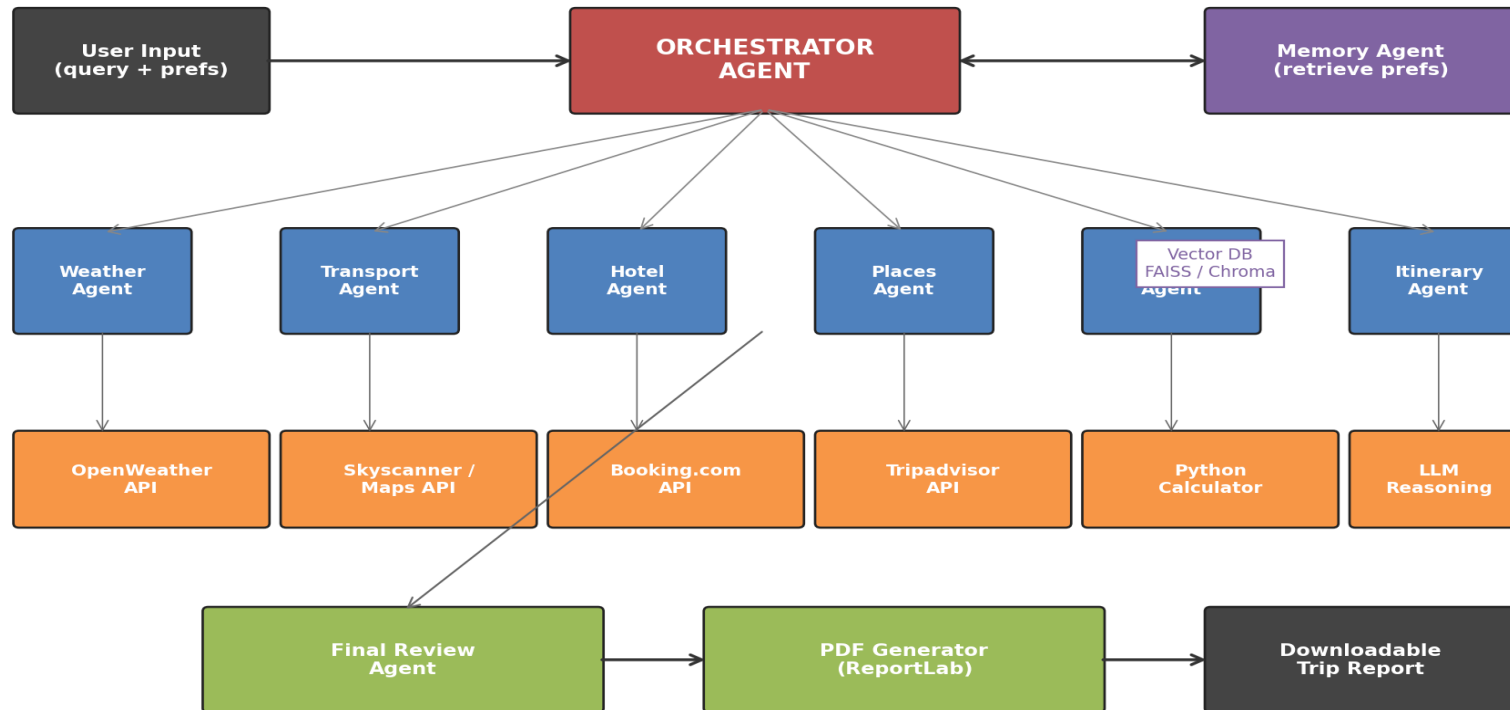
Presented: May 2026

One query in → 9 AI agents working together → professional PDF out



Orchestrator at the centre – 9 specialist agents around it

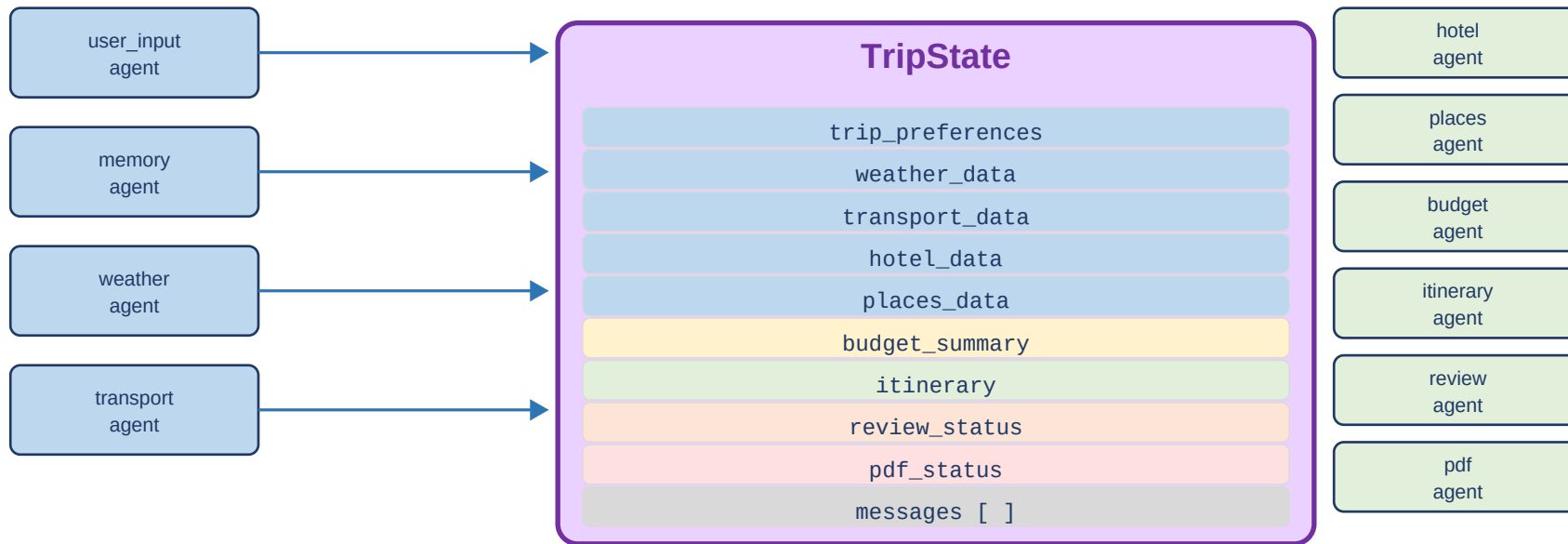
Multi-Agent Trip Planner – System Architecture



Red box	Orchestrator – the only decision-maker; routes everything
Blue boxes	4 parallel specialists – Weather, Transport, Hotel, Places
Orange boxes	Real APIs these tools would call in production
Green boxes	Final Review – PDF Generator – Downloadable Report

Whiteboard

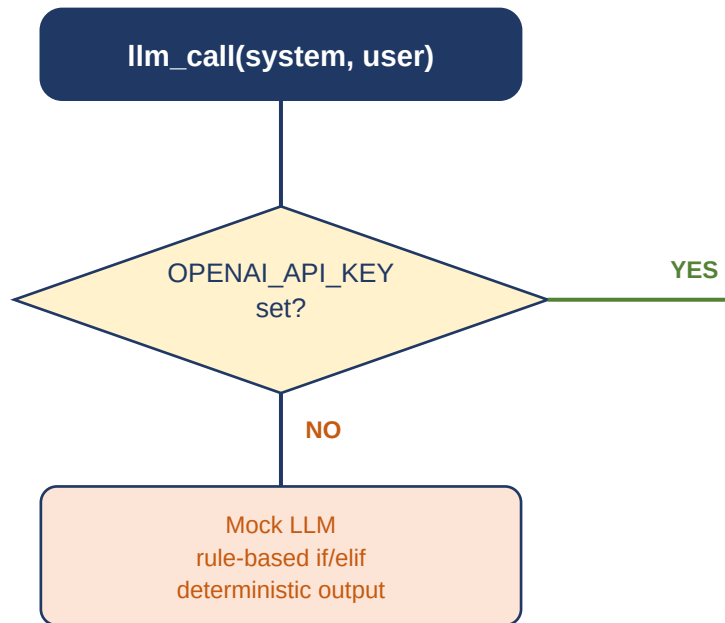
All 9 agents read from and write to one single TypedDict



Every agent reads from state – writes its own key – never touches other keys

With or Without OpenAI

One function hides the real/mock switch from all agents



Mock LLM behaviour

Prompt contains	Mock returns
"itinerary"	Day 1-5 plan text
"review"	{"approved": true}
"packing"	sunscreen; swimsuit;
"summary"	A relaxed 5-day trip
anything else	Acknowledged.

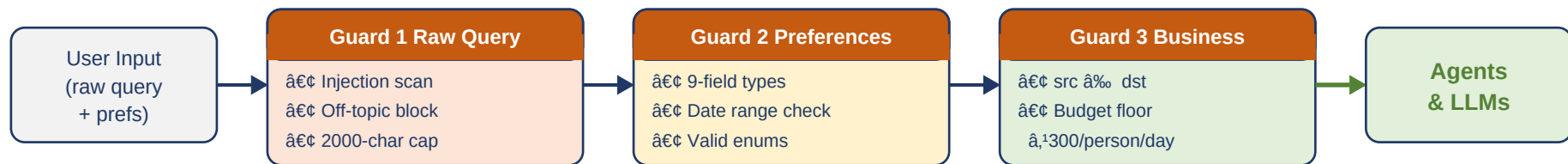
“ Produces a real PDF with zero API costs

“ Same output every run (seed=42)

Input Validation

Every input is validated BEFORE any agent or LLM sees it

Bonus: LLM Input Sanitiser "strips" control chars, caps length before every llm_call()



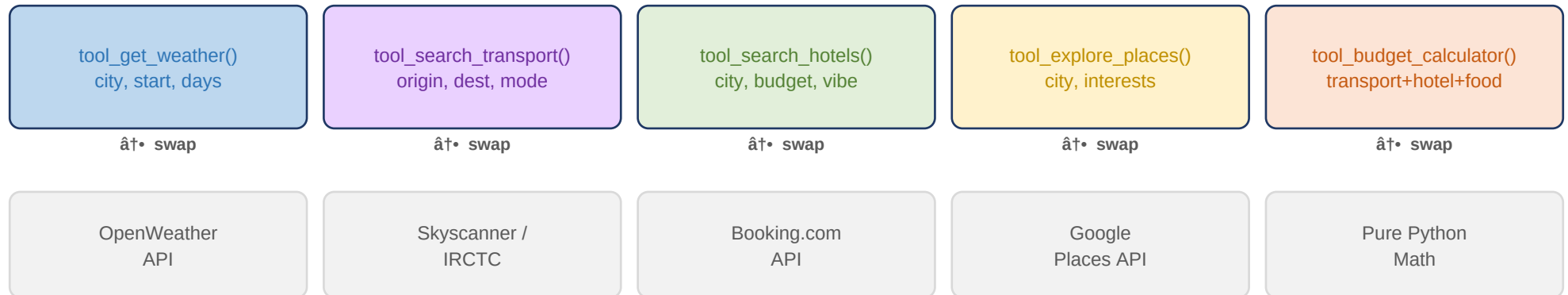
— Any guard fails 'InputGuardrailError' raised 'orchestrator handles gracefully'

Guard	What it blocks	Example that fails
Guard 1 " Raw Query	Prompt injection; off-topic; >2000 chars	"Ignore all previous instructions"
Guard 2 " Preferences	Wrong types; bad dates; unknown enums	start_date="yesterday", days="lots"
Guard 3 " Business	Source = destination; budget too low	source="Goa", dest="Goa" or budget=<1100 for 5 days

appable)

Each tool is a Python function with # TODO: real API â€” swap any time

Mock tools (used now â€” deterministic, seed=42)



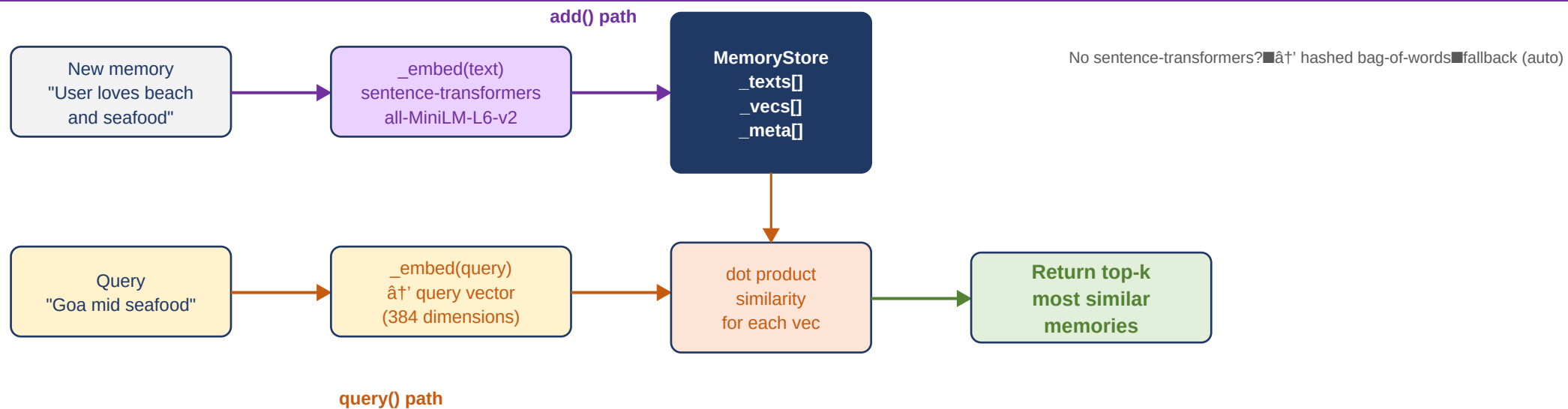
Real APIs (swap in production â€” interface stays the same)

Why mock tools?

â€” Zero API cost â€” Same output every run (seeded) â€” No rate limits â€” Works offline

tic Vector Database

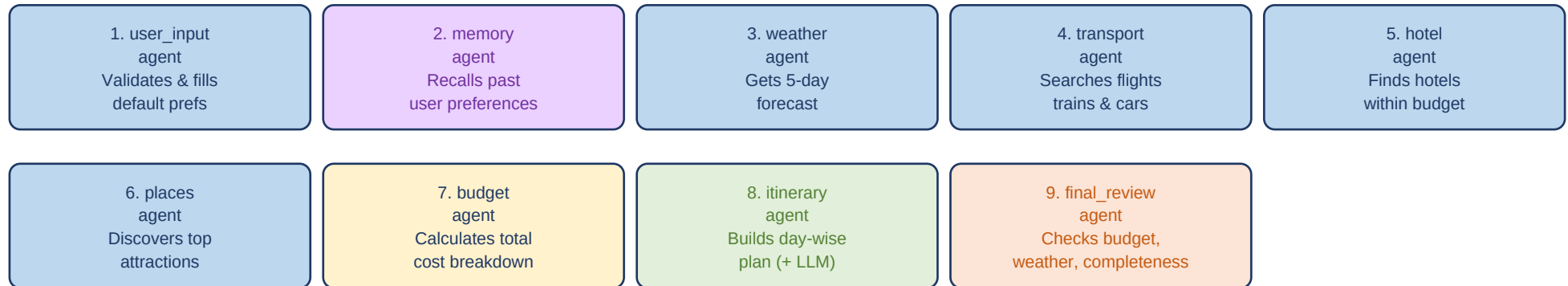
Remembers user preferences across trips using cosine similarity



In this notebook	In production
NumPy-based MemoryStore (30 lines)	Chroma / FAISS / Pinecone
sentence-transformers (local model)	OpenAI text-embedding-3-small
In-process (lost on restart)	Persistent disk/cloud storage

e Job Each

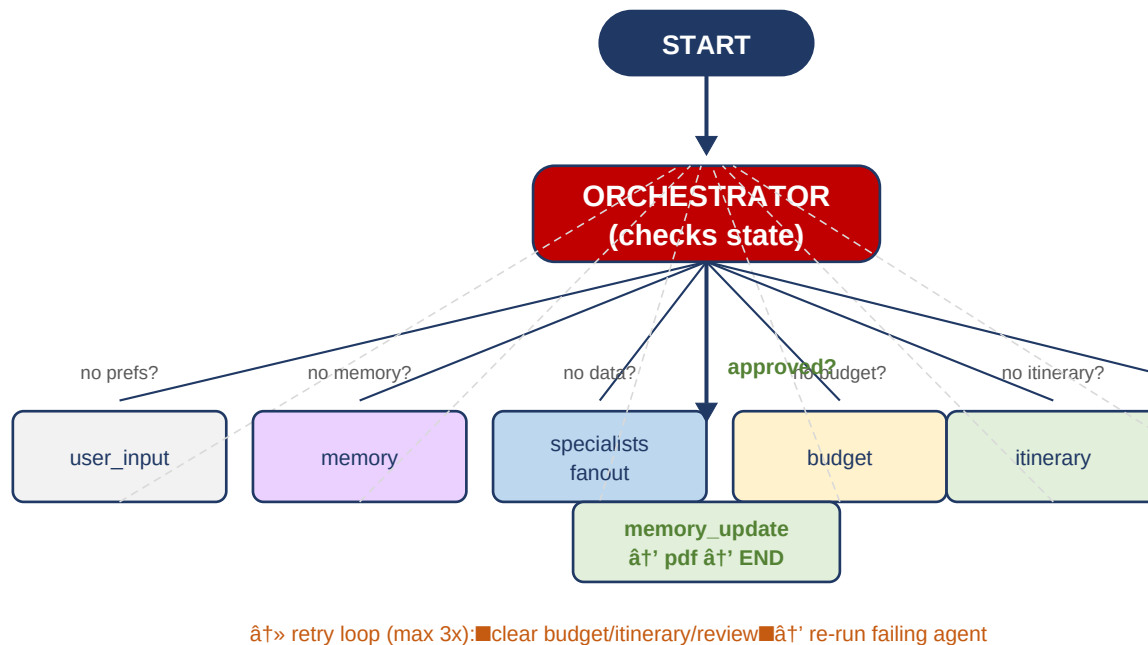
Each agent: read state â†’ call tool â†’ return updated keys only



Pattern: `def agent(state: TripState) â†’ dict Â· Read â†’ Tool â†’ Return {key: value}`

âš¡ 4 agents run in
PARALLEL

Weather, Transport, Hotel, Places all fire at the same time via the `specialists_fanout` node. They write to different state keys â€” no collision. LangGraph waits for all 4 before proceeding.



Why rule-based?

Transparent, debuggable, deterministic.

Swap for LLM routing?

Yes → replace the if/elif with:
`llm_call(SUPERVISOR_PROMPT, state)` Same graph works identically.

Retry logic

If review fails:

→ Over budget → retry hotel agent

→ Too rainy → retry places agent

→ Transport issue → retry transport

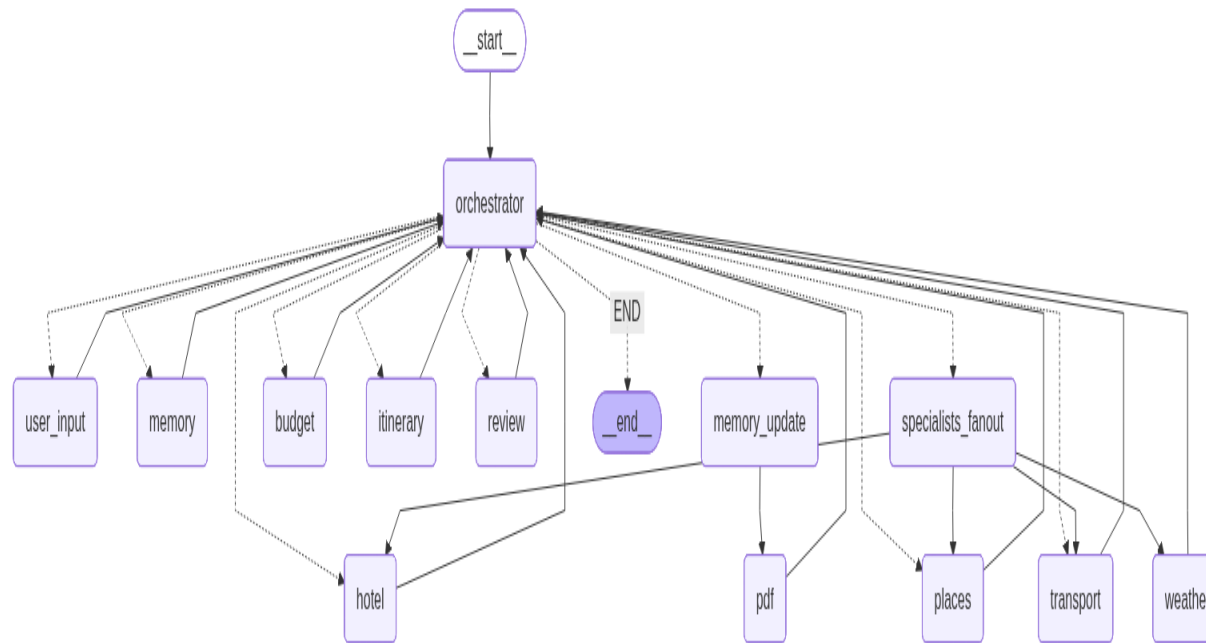
→ Gives up after 3 retries

Checkpointing

MemorySaver saves full state after every step → crash recovery & replay.

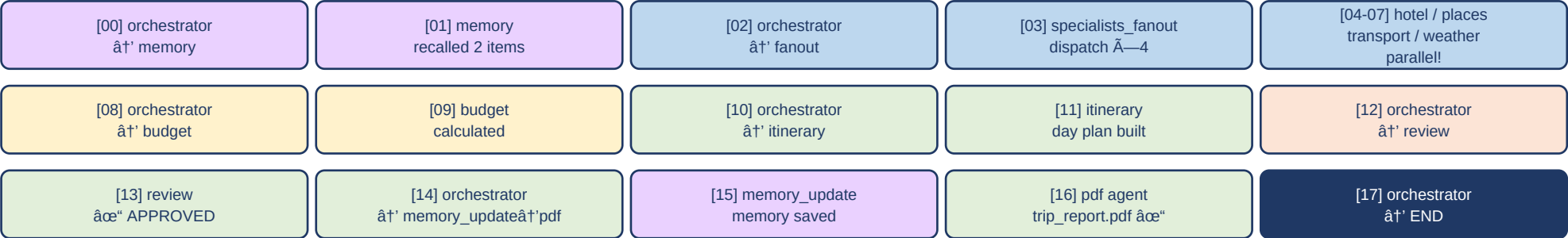
The Compiled Graph

13 nodes • fan-out parallelism • conditional edges • MemorySaver check



Concept	What it means in this graph	Code
Fan-out	4 specialists run in parallel – one edge from fanout to each	<code>g.add_edge("specialists_fanout", "weather")</code> –4
Conditional edge	Orchestrator decides the next node each pass	<code>g.add_conditional_edges("orchestrator", route_fn, {...})</code>
MemorySaver	Full state saved after every step – crash-safe & resumable	<code>app = g.compile(checkpointer=MemorySaver())</code>
thread_id	Isolates each trip planning session	<code>config={"configurable": {"thread_id": "..."} }</code>

Input: "5-day Goa trip from Bangalore, ₹1,30,000, couple, flight"



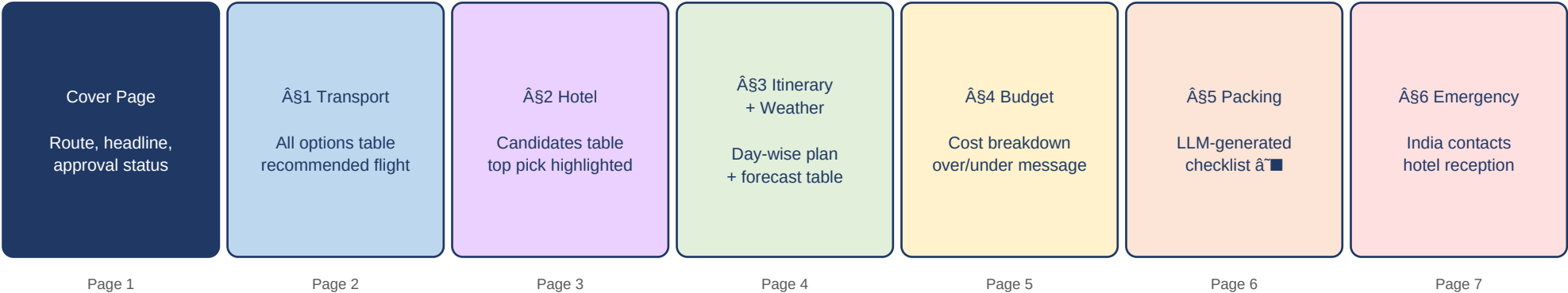
Total runtime: < 5 seconds • Output: trip_report.pdf

Output	Value
Hotel	Backwoods Hostel (fits ₹130k budget)
Transport	IndiGo flight â€” ₹14,500
Total cost	₹126,800 (under ₹130,000 âœ“)
Headline	A relaxed 5-day couple's beach trip to Goa with seafood and nightlife.
Review	âœ“ APPROVED â€” 0 issues
PDF	trip_report.pdf â€” 6 sections, A4 format

tion Trip Report

ReportLab builds a professional A4 PDF from the final state

Each section draws data directly from the final TripState – no re-computation



Generated with ReportLab – A4 format – Available at: [trip_report.pdf](#)

Agent reads from state	Writes to PDF
trip_preferences	Cover: route, days, travelers, budget
itinerary["headline"]	Cover: AI-generated headline
transport_data	Section 1: All options + recommended
hotel_data	Section 2: Candidates + top pick
itinerary + weather_data	Section 3: Day plan + forecast table
budget_summary	Section 4: Breakdown table + delta message
llm_call(packing prompt)	Section 5: Packing checklist items

Quality Evaluation

4 rule-based metrics (no API key) + 2 LLM-based metrics (with OpenAI key)

Rule-based metrics (no API needed)				LLM-based (needs key)	
BudgetAdherence Metric Total cost ≤ stated budget?	PreferenceMatch Metric Interests in itinerary text?	Itinerary Completeness Days + headline + hotel + transport?	Review Approval final_review approved?	AnswerRelevancy (OpenAI) Itinerary relevant to query?	Faithfulness (OpenAI) No hallucinations in output?
Score: 1.00 [PASS]	Score: 0.67 [PASS]	Score: 1.00 [PASS]	Score: 1.00 [PASS]	Score: 1.0 [LLM]	Score: 1.0 [LLM]

TC-1: Goa trip → 4/4 rule-based PASS • TC-2: Your trip → 3/4 PASS (Preference Match failed → real finding!)

🔴 TC-2 Preference Match FAIL is a REAL finding

Bangalore trip interests = ["nature", "party"] → neither word appeared in the itinerary output text. This tells us the itinerary_agent needs to reference user interests more explicitly when attraction data is sparse. **Evaluation found a real bug.**

Assignment Deliverables “

- “ Architecture diagram
- “ LangGraph workflow
- “ Agent design (9 agents)
- “ Orchestrator logic
- “ Tool design (5 tools)
- “ Memory system
- “ PDF generator
- “ Working demo
- “ Input Guardrails
- “ DeepEval evaluation

Section 14 “ 4+2 metrics, 2 test cases

Production Upgrades (Easy)

Upgrade	What to change
Real Weather	Replace <code>tool_get_weather()</code> â†’ <code>OpenWeatherMap API</code>
Real Flights	Replace <code>tool_search_transport()</code> â†’ <code>Skyscanner/Amadeus</code>
Real Hotels	Replace <code>tool_search_hotels()</code> â†’ <code>Booking.com API</code>
Real LLM	Set <code>OPENAI_API_KEY</code> â†’ <code>gpt-4o-mini auto-activated</code>
Real Vector DB	Swap <code>MemoryStore</code> class â†’ <code>Chroma / FAISS / Pinecone</code>
LLM Routing	Replace <code>if/elif</code> â†’ <code>llm_call(SUPERVISOR_PROMPT, state)</code>

Key pattern: Agents are pure functions. Tools are swappable. State is typed. Graph is compiled. All production patterns “ just mock data.